

AWS Developer Certification – Domain 1 Study Summary

Domain 1: Development with AWS Services — Task Statement 2: *Develop Code for AWS Lambda*

Continuing the Domain 1 series. Practice questions from the transcript appear as blockquotes, with a quick-reference answer key at the end.

Foundations to Review

- Two Well-Architected Framework principles stand out for Lambda: **Operational Excellence** → treat operations as code; **Reliability** → scale horizontally to increase aggregate workload availability.
- Lambda is a serverless compute service: each invocation runs inside an **execution environment** with a runtime matching your function's language. The environment manages the resources your function needs and handles lifecycle support for the runtime and any extensions.
- Function configuration covers memory, execution timeout, and more — dive deeper into the execution environment's lifecycle.

Invocation & Configuration

- Triggers come from lifecycle events, external requests, AWS services, schedules, or manual invocation — each is synchronous, asynchronous, or polling-based.
- Configurable parameters: memory, concurrency, timeout, runtime, handler, layers, extensions, triggers, and destinations.
- **Reserved concurrency** splits the shared concurrency pool into subsets, capping how much of it any single function can consume — useful for cost control and to stop one function starving another.

Practice question — two identical functions, different performance: Two Lambda functions with identical code process ad-hoc requests, but CloudWatch shows one consistently taking longer than the other.

Answer: Lambda's unit of scale is **concurrent execution**. Fix it by either raising the concurrency limit for both functions, or lowering the limit on the faster function so it stops crowding out the slower one's share of concurrency.

Lambda with Streams

- For functions processing **Kinesis** or **DynamoDB Streams**, the **number of shards is the unit of concurrency** — 100 active shards means at most 100 concurrent invocations, since each shard's events are processed in sequence.

Practice question — same function, different table per API stage: One Lambda function backs multiple stages of a REST API, but must read from a different

DynamoDB table depending on the stage. Environment variable or stage variable?

Answer: stage variable — a configuration attribute set on the API's deployment stage.

An environment variable alone won't do it; you integrate Lambda with API Gateway via the stage variable plus the right mapping configuration.

Event Source Mapping

To process items arriving from a stream or queue, configure an **event source mapping** — a Lambda resource that reads from an SQS queue, Kinesis stream, or DynamoDB stream and delivers items to your function in batches. It also handles retries from errors or throttling on your behalf.

Error Handling

Two categories of error can occur on invocation:

- **Invocation errors** — request, caller, account
- **Function errors** — function, runtime

Your strategy for either: retry, send the event to a queue for debugging, or ignore it. If you retry, your code must be able to handle the same event arriving more than once without creating duplicates or side effects.

Asynchronous invocations

- Lambda manages retries itself and can route invocation records to a destination — **SQS, SNS, Lambda, or EventBridge** — with separate destinations configurable for success vs. failure.
- A **dead-letter queue (DLQ)** adds further control over message handling across async sources (S3, SNS, IoT, and others).
- **Lambda retries a function error twice.** If a function can't keep up with incoming volume, events may sit queued for hours or days; a DLQ on the function captures anything that never gets successfully processed.
- For event source mappings reading from a queue specifically, control retry timing and where failed events land via the **visibility timeout** and **redrive policy** on the source queue.

Synchronous vs. asynchronous invocation

- Synchronous: Lambda runs the function and waits, then returns the response plus metadata like the function version invoked. Trigger this via the CLI with the `invoke` command.
- Asynchronous: services like **S3 and SNS** hand off the event without waiting; Lambda takes it from there, including optional downstream chaining via destinations.

Observability

Combine **logs, metrics, alarms, and tracing** to catch and diagnose issues quickly — **CloudWatch** and **AWS X-Ray** are the go-to services for this.

VPC Access

- To reach private VPC resources, Lambda assigns your function an **ENI (Elastic Network Interface)** with a **security group** for each subnet in its VPC configuration.
- Multiple functions can share one ENI only if they share the same subnet and security group. Lambda uses your function's own permissions to create and manage the ENI automatically.
- Lambda-specific IAM condition keys for extra control over VPC settings:
 - `lambda:VpcIds` — allow/deny specific VPCs
 - `lambda:SubnetIds` — allow/deny specific subnets
 - `lambda:SecurityGroupIds` — allow/deny specific security groups

Testing

- **Unit testing** — call an entry-point function that invokes others, testing them together as a single unit.
- **Integration testing** — test how services interact with one another.
- Testing a Lambda function means testing its logic, payload, and invocation — include end-to-end coverage.
- As your app scales, lean on automated tests, and automate testing within your build pipeline.

Performance Tuning: Cold Starts

- On a fresh invocation, Lambda prepares an execution environment: it downloads your code (from an internal S3 bucket, or ECR for container-packaged functions), builds the environment with the specified memory/runtime/configuration, runs any initialization code, then runs the handler.
- You're **not billed** for this setup time, but it does add latency to the invocation.
- Afterward, the environment is frozen and can be reused for the next invocation — a faster **warm start**. Cold starts show up more often in dev/test functions.
- Don't rely on environment reuse for performance: updating your code or changing the function's configuration forces the next invocation to be cold again.
- Scheduling dummy invocations (e.g., an EventBridge rule every minute) to keep things "warm" **doesn't reliably help** — it won't cover instances created by scale-out, or invocations landing in a different Availability Zone.
- **Better fix: provisioned concurrency**, which keeps environments pre-initialized for the lowest possible latency.

Performance Tuning: Memory & Compute

- Increasing a function's memory also increases its CPU allocation — worth adjusting if your function is CPU- or memory-bound.
- Picking the right memory setting balances speed against cost; automate this rather than manually testing allocations.
 - **AWS Lambda Power Tuning tool** — uses Step Functions to run your function concurrently across multiple memory allocations and compare performance.
 - **Lambda Cost Optimizer** — automates cost/performance analysis across all your functions.
- Also look at optimizing **static initialization**, since it contributes to overall cold-start time.
- In a distributed system, request duration varies with network delays, traffic, message retries, failovers, and each service's own performance profile — **AWS X-Ray** helps pinpoint the outliers.

When Not to Use Lambda

- **Orchestration-heavy functions** (calling and coordinating other services) create idle time and added cost — move that logic to **AWS Step Functions** for a more maintainable, resilient (and often cheaper) state machine.
- **Pass-through functions** that move data with no real business logic can often be replaced with **direct service integrations** — e.g., skip the Lambda between API Gateway and DynamoDB and use **Velocity Template Language (VTL)** in an API Gateway service integration instead, improving scalability and cutting cost.
- **Broad filtering functions** that only act on a small subset of incoming events should filter *before* invoking Lambda:
 - S3 events → filter by prefix/suffix
 - SNS → filter messages before invoking targets
 - EventBridge → use content-filtering rules
 - This cuts cost and ensures Lambda only runs for events your application actually cares about.

Broader List to Dive Deeper Into

Execution context reuse, environment variables, handling dependencies in function code, shrinking deployment packages, memory tuning, execution timeout, and monitoring Lambda logs.

Up Next

Task Statement 3: Use data stores in application development.

Quick-Reference Answer Key

#	Scenario / Question	Answer
1	Two identical functions, one processes slower per CloudWatch	Adjust concurrency limits — raise both, or lower the faster one's limit
2	One function needs a different DynamoDB table per API stage	Stage variables, not environment variables
3	Reading items from an SQS/Kinesis/DynamoDB stream into a function	Event source mapping
4	How many times does Lambda retry a function error?	Twice
5	AWS CLI command to invoke a function synchronously	<code>invoke</code>
6	What Lambda assigns per VPC subnet to reach private resources	An ENI with a security group
7	Best long-term fix for cold-start latency	Provisioned concurrency (not warm-up pings)